BCS-602	Compiler Design	L T P 3 1 0	3 Credits

Pre-requisites: Basic knowledge of theory of computation and compilers.
Course Objectives: To provide a thorough understanding of the internals of Compiler Design

CO1	To discuss the techniques of scanning , parsing & semantic elaboration well enough to build or modify front end.
CO2	To expose the critical issues in modern compilers & provide them with the background to tackle those problems
CO3	Provide an understanding of the fundamental principles in compiler design
CO4	Learn the process of translating a modern high-level language to executable code required for compiler construction
CO5	Provide the skills needed for building compilers for various situations that one may encounter in a career in Computer Science.

Detailed Syllabus

MODULE-I

Introduction to Compiler: Phases and passes, Bootstrapping, Finite state machines and regular expressions and their applications to lexical analysis, implementation of lexical analyzers, lexical-analyzer generator, LEX-compiler. Lexical Phase errors: detection &recovery. Formal grammars and their application to syntax analysis, BNF notation, ambiguity. The syntactic specification of programming languages: Context free grammars, derivation and parse trees, capabilities of CFG.Basic Parsing Techniques: Parsers, Shift reduce parsing, operator precedence parsing, top down parsing, predictive parsers.

MODULE-II

Automatic Construction of efficient Parsers: LR parsers, the canonical Collection of LR (0) items, constructing SLR parsing tables, constructing Canonical LR parsing tables, Constructing LALR parsing tables, using ambiguous grammars, an automatic parser generator, implementation of LR parsing tables. Syntactic phase errors – detection & recovery. Syntax-directed Translation: Syntax-directed Translation schemes, Implementation of Syntax directed Translators, Intermediate code, postfix notation, Parse trees & syntax trees, three address code, quadruple & triples, translation of assignment statements, Boolean expressions, statements that alter the flow of control, postfix translation, translation with a top down parser. More about translation: Array references in arithmetic expressions, procedures call, declarations and case statements. Semantic errors – detection & recovery.

MODULE-III

Symbol Tables: Data structure for symbols tables, representing scope information. RunTime Administration: Implementation of simple stack allocation scheme, storage allocation in block structured language. Code Generation: Design Issues, the Target Language. Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, Code

Generator.Code optimization: Machine-Independent Optimizations, Loop optimization, DAG representation of basic blocks, value numbers and algebraic laws, Global Data-Flow analysis

Text Books:

1. Aho, Sethi& Ullman, "Compilers: Principles, Techniques and Tools", Pearson Education.

Reference Books:

2. Raghvan, " Principles of Compiler Design", TMH
3. Kenneth Louden," Compiler Construction", Cengage Learning.
4. Charles Fischer and Ricard LeBlanc, "Crafting a Compiler with C", Pearson Education

Course Outcomes: After the completion of the course the student will be able to:

CO1	Understand the basic concepts and application of Compiler Design
CO2	Apply their basic knowledge Data Structure to design Symbol Table, Lexical Analyzer
CO3	Understand and Implement a Parser -Top Down and Bottom Up Design
CO4	Understand strength of Grammar and Programming Language
CO5	Understand the concept of Code generation